EXPERIMENT 1

Write a Python Program to Solve the 8-Puzzle Problem

Aim

To develop a Python program that solves the **8-Puzzle Problem** using the **A* Search Algorithm** and finds the shortest path from the initial state to the goal state.

Objective

- To understand state-space search problems.
- To apply the A* Search Algorithm in Artificial Intelligence.
- To find the optimal sequence of moves to reach the goal state.
- To learn heuristic-based search techniques.

Algorithm

Step 1: Define the initial state and goal state of the puzzle.

Step 2: Calculate the heuristic value (Manhattan Distance or Misplaced Tiles).

Step 3: Place the initial state into the priority queue.

Step 4: Remove the state having the lowest **f(n) = g(n) + h(n)** value.

Step 5: Check whether the current state is the goal state.

- If Yes, stop.
- Otherwise continue.

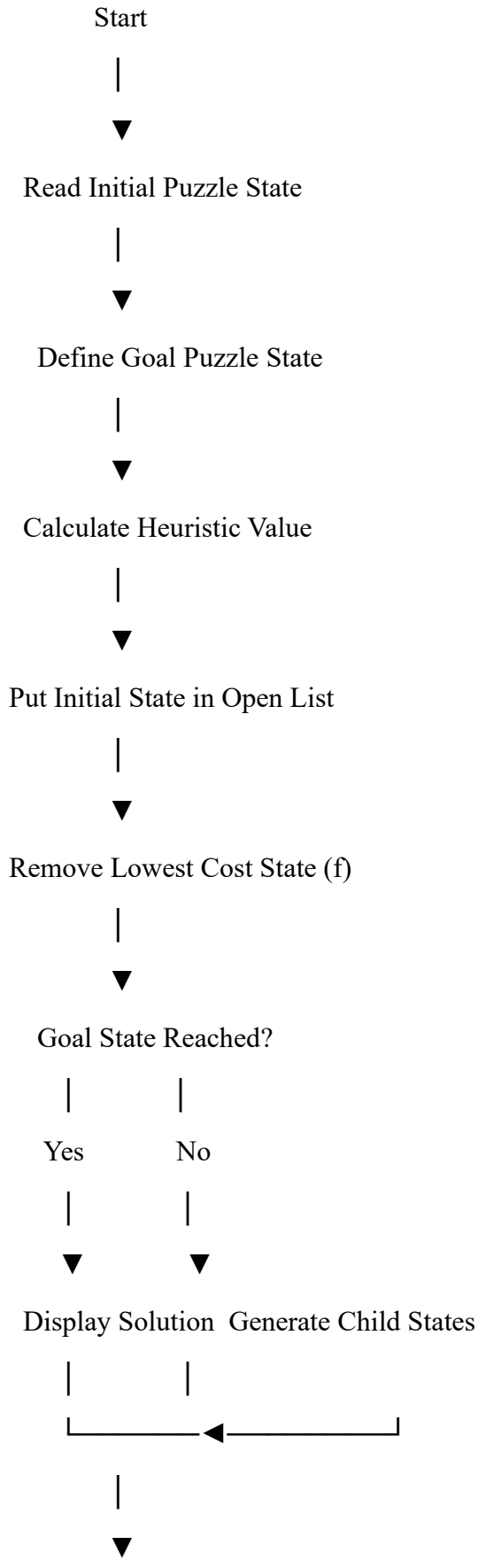
Step 6: Generate all possible child states by moving the blank tile (Up, Down, Left, Right).

Step 7: Calculate the cost of each child state.

Step 8: Insert unexplored child states into the priority queue.

Step 9: Repeat Steps 4–8 until the goal state is reached.

Flowchart


Start
Read Initial Puzzle State
Define Goal Puzzle State
Calculate Heuristic Value
Put Initial State in Open List
Remove Lowest Cost State (f)
Goal State Reached?
Yes
No
Display Solution
Generate Child States

Stop

Python Program

```
from queue import PriorityQueue


goal = [[1,2,3],
        [4,5,6],
        [7,8,0]]


class Puzzle:
    def __init__(self,state,parent,move,depth):
        self.state=state
        self.parent=parent
        self.move=move
        self.depth=depth

    def heuristic(self):
        count=0
        for i in range(3):
            for j in range(3):
                if self.state[i][j]!=0 and self.state[i][j]!=goal[i][j]:
                    count+=1
        return count

    def cost(self):
        return self.depth+self.heuristic()

    def __lt__(self,other):
        return self.cost()<other.cost()
```

```
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j]==0:
                return i,j

def generate(node):
    x,y=find_blank(node.state)
    moves=[(-1,0),(1,0),(0,-1),(0,1)]
    children=[]

    for dx,dy in moves:
        nx=x+dx
        ny=y+dy

        if 0<=nx<3 and 0<=ny<3:
            new=[row[:] for row in node.state]
            new[x][y],new[nx][ny]=new[nx][ny],new[x][y]
            children.append(Puzzle(new,node,(nx,ny),node.depth+1))

    return children

start=[[1,2,3],
       [4,0,6],
       [7,5,8]]

pq=PriorityQueue()
pq.put(Puzzle(start,None,None,0))
```

```
visited=[]

while not pq.empty():

    current=pq.get()

    if current.state==goal:
        path=[]

        while current:
            path.append(current.state)
            current=current.parent

        path.reverse()

        print("Solution Path:\n")

        for state in path:
            for row in state:
                print(row)
            print()

        break

    visited.append(current.state)

    for child in generate(current):
        if child.state not in visited:
```

```
        pq.put(child)
```

Sample Output

Solution Path:

[1, 2, 3]

[4, 0, 6]

[7, 5, 8]

[1, 2, 3]

[4, 5, 6]

[7, 0, 8]

[1, 2, 3]

[4, 5, 6]

[7, 8, 0]

Result

The Python program successfully solved the **8-Puzzle Problem** using the **A* Search Algorithm** and generated the optimal sequence of moves from the initial state to the goal state.

Conclusion

The **8-Puzzle Problem** is a classical Artificial Intelligence search problem. The A* Search Algorithm efficiently finds the shortest path by combining the actual path cost and heuristic value. It reduces unnecessary state exploration and guarantees an optimal solution when an admissible heuristic is used. This experiment demonstrates the practical application of heuristic search techniques in AI.

C: > Users > Varshini M > OneDrive > Desktop > Here!!! > ARTIFICIAL INTELLIGENCE > VSC EXPs > EXP1.py > ...

```python
from queue import PriorityQueue

goal = [[1,2,3],
        [4,5,6],
        [7,8,0]]

class Puzzle:
    def __init__(self,state,parent,move,depth):
        self.state=state
        self.parent=parent
        self.move=move
        self.depth=depth

    def heuristic(self):
        count=0
        for i in range(3):
            for j in range(3):
                if self.state[i][j]!=0 and self.state[i][j]!=goal[i][j]:
                    count+=1
        return count

    def cost(self):
        return self.depth+self.heuristic()

    def __lt__(self,other):
        return self.cost()<other.cost()

def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j]==0:
                return i,j

def generate(node):
    x,y=find_blank(node.state)
    moves=[(-1,0),(1,0),(0,-1),(0,1)]
    children=[]

    for dx,dy in moves:
        nx=x+dx
        ny=y+dy

        if 0<=nx<3 and 0<=ny<3:
            new=[row[:] for row in node.state]
            new[x][y],new[nx][ny]=new[nx][ny],new[x][y]
            children.append(Puzzle(new,node,(nx,ny),node.depth+1))

    return children

start=[[1,2,3],
       [4,0,6],
       [7,5,8]]

pq=PriorityQueue()
pq.put(Puzzle(start,None,None,0))

visited=[]

while not pq.empty():

    current=pq.get()

    if current.state==goal:
        path=[]

        while current:
            path.append(current.state)
            current=current.parent

        path.reverse()

        print("Solution Path:\n")

        for state in path:
            for row in state:
                print(row)
            print()

        break

    visited.append(current.state)

    for child in generate(current):
        if child.state not in visited:
            pq.put(child)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Varshini M\AppData\Local\Programs\Microsoft VS Code> & "C:\Users\Varshini M\AppData\Local\Programs\Python\Python314\py
i M/OneDrive/Desktop/Here!!!/ARTIFICIAL INTELLIGENCE/VSC EXPs/EXP1.py"

[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

[1, 2, 3]
[4, 5, 6]
[7, 8, 0]

PS C:\Users\Varshini M\AppData\Local\Programs\Microsoft VS Code>
```